

Construindo Aplicações com LLMs

Workshop Evoluum

- Não é tutorial · dicas reais
- Mão na massa · interativo
- Sem bullshit 🎉

O que veremos hoje

- Agentes de IA? O que é o Hype? 
- Fundamentos do Prompt: A base de tudo 
- A “Mágica” dos Agentes: Structured Outputs ✨
- Contexto é Rei: Controle a Janela de Contexto 
- Workflows > Agentes: Comece Simples 
- MCP: Catálogo de Ferramentas 
- Codando com Cursor 
- Construção ao Vivo & Perguntas  

O que é um **AI Agent**?

“Compartilhe: Já usou um LLM? Onde?”

Fundamentos de Prompting - Zero/One/Few-Shot

- Zero-shot: Só descreve, sem exemplo

Classifique como positivo ou negativo.

- One/few-shot: Exemplos guiam o padrão

```
// Few-shot pra classificar sentimento
```

```
{"input": "Dia incrível!", "output": "positivo"}
```

```
{"input": "Que droga!", "output": "negativo"}
```

Exemplos Few-Shot: Bom ou Ruim?

1: (Ruim - Pouca diversidade)

```
{"input": "Quero falar com o suporte agora!", "intent": "suporte"}  
{"input": "Preciso de ajuda técnica urgente!", "intent": "suporte"}  
{"input": "Alguém do suporte pode me atender?", "intent": "suporte"}  
{"input": "Gostaria de adquirir a versão Pro.", "intent": "vendas"}
```

2: (Bom - Casos variados)

```
{"input": "Meu app trava no login.", "intent": "suporte"}  
{"input": "Não consigo enviar e-mails.", "intent": "suporte"}  
{"input": "Quero o plano Premium hoje.", "intent": "vendas"}  
{"input": "Como é o licenciamento?", "intent": "vendas"}  
{"input": "O sistema não carrega configs.", "intent": "suporte"}
```

Fundamentos de Prompting - System/Context/Role

- System/context/role: Define quem o LLM é
- Exemplo:

```
// System prompt pra guia turístico
```

```
System: Você é um guia turístico engraçado.
```

```
Context: Sugira 3 lugares no Rio.
```

Fundamentos de Prompting - Chain of Thought

Chain of Thought (CoT): Raciocínio passo a passo

Exemplo de verdade (prioridade & escalonamento de ticket):

Critérios:

- Plano → Free (72 h) · Pro (48 h) · Enterprise (24 h)
- Severidade → blocker > major > minor
- Regras → Se violou SLA ou é blocker ⇒ escalar

Solicitação:

Ticket #8421 (cliente Enterprise) aberto há 30 h.

Usuário relata: 'Não consigo finalizar pagamentos, tudo falha!'

Exemplo de Chain of Thought

Prompt

Decida se deve escalar para Engenharia. Pense passo a passo e depois responda apenas
JSON{ "escalar": true/false, "motivo": "<texto curto>" }

Raciocínio esperado:

- 1) Plano = Enterprise $\Rightarrow$ SLA 24 h
- 2) Tempo aberto = 30 h $\Rightarrow$ violou SLA
- 3) Severidade = blocker (pagamentos indisponíveis)
- 4) Qualquer critério já bastaria; temos ambos

Resposta: {"escalar": true, "motivo": "SLA violado e blocker"}

Fundamentos de Prompting - Structured Output

JSON para structured output

- e.g. Geração de histórias de usuário a partir de transcript de call):

Prompt (resumido):

Você é Product Owner sênior. Abaixo segue o transcript da reunião de discovery <transcript>.

Gere até 5 histórias de usuário priorizadas em formato JSON:

```
[{ "ator": <string>, "acao": <string>, "valor": <string>, "sla_horas": <int> }].
```

Priorize pelo impacto mencionado na call.

Primeiro pense passo a passo e destaque a motivacao de suas escolhas na chave justificativa.

Retorne estritamente:

```
{  
  "justificativa": "<passo a passo curto>"  
  "historias": [...],  
}
```

Resposta esperada (exemplo):

```
{
  "historias": [
    { "ator": "Comprador", "acao": "finalizar pagamento com 1 clique", "valor": "reduzir abandono",
      "sla_horas": 120 },
    { "ator": "Admin", "acao": "gerar relatório de vendas diário", "valor": "tomar decisões rápidas",
      "sla_horas": 72 }
  ],
  "justificativa": "Pagamentos e visibilidade de vendas foram tópicos mais críticos na call."
}
```

Dica: Use justificativa como CoT, um campo str aberto para o LLM, e no backend você parseia somente historias. Nem sempre necessario, mas pode ajudar em tarefas mais complexas.

Fundamentos de Prompting - Parâmetros

- Parâmetros: Temperature, Top-K, Top-P
- Exemplo:

```
// Upsell criativo  
{"temperature":0.9,"top_p":0.9}  
// Contrato jurídico  
{"temperature":0.2,"top_p":0.5}
```

Structured Outputs ✨

- Segredo dos agentes: Saídas estruturadas
- Nada de texto solto: use JSON pro código entender
- Tool calling? Só um structured output chique
- Exemplo (DevOps):

```
{  
  "name": "list_git_tags",  
  "arguments": {"repo": "backend-api"}  
}
```

Controle a Janela de Contexto

- LLMs são **stateless**: Sem memória
- Contexto é TUDO que o LLM vê
- Engenharia de contexto tao importante quanto engenharia de prompt
- Exemplo padrão (OpenAI):

```
[  
  {"role": "user", "content": "Deploy o backend"},  
  {"role": "assistant", "tool_calls": [  
    {"id": "1", "name": "list_git_tags"}  
  ]}  
]
```

Contexto Rico: Faça do Seu Jeito

- Contexto otimizado:

```
<slack_message from="@alex" channel="#deployments">  
  Deploy o backend?  
</slack_message>  
<tool_executed name="list_git_tags">  
  <result><tags><tag>v1.2.3</tag></tags></result>  
</tool_executed>  
Próximo passo?
```

- Você decide como o LLM “enxerga” o contexto, use ao seu favor

Chamada de API Básica

```
from openai import OpenAI
client = OpenAI(api_key="...")
messages = [
    {"role": "system", "content": "Você é analista de CX."},
    {"role": "user",
     "content": "Um ticket aberto há 5 dias tem SLA de 3 dias. Está violado? Responda Sim/
Não."}
]
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages
)
print(response.choices[0].message.content)
```

Chamada Estruturada (Python)

```
async def call_structured[T: BaseModel](
    self, model: str, system_prompt: str,
    user_prompt: str, output_model: type[T]
) -> T:
    response = await self.client.beta.chat.completions.parse(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        response_format=output_model,
    )
    if parsed := response.choices[0].message.parsed:
        return parsed
    raise ValueError("No JSON? Algo deu ruim!")
```


Chamada Estruturada (Typescript)

```
export async function callStructured<T>(  
  model: string,  
  systemPrompt: string,  
  userPrompt: string,  
  schema: ZodType<T>  
) : Promise<T> {  
  const response = await openai.responses.parse({  
    model,  
    input: [  
      { role: "system", content: systemPrompt },  
      { role: "user", content: userPrompt },  
    ],  
    text: { format: zodTextFormat(schema, "response") },  
  });  
  if (!response.output_parsed)  
    throw new Error("Sem JSON! Dá um log aqui.");  
}
```

Construindo Aplicações com LLMs

```
    return response.output_parsed;  
}
```

Comece Simples, Mantenha o Controle 🌱

- **Workflow:** Fluxo previsível, você manda
- **Agent:** LLM escolhe o próximo passo
- Dica: Workflow primeiro, agente só se precisar
- Faz funcionar antes de dar liberdade

MCP: Catálogo de Ferramentas

- Descubra e chama ferramentas
- Integra com workflows ou agents
- Demo: Lista ferramentas → chama uma

Dicas de Projetos Reais

- Controle o **context window**
- Ferramentas = **structured outputs**
- **Human-in-loop** pra não dar ruim
- Modelos menores + router > modelo grandão

Codificando com Cursor 🤖

- Chat inline, abas de agentes, regras
- **Trust-but-verify**: Revisar é rei
- Demo: Cursor na prática

Ideias pra Agentes

- Que problema você quer resolver com um agent?
- Discussão: 10 minutos
- Escolhemos uma ideia juntos

Mão na Massa: Agente Simples 🤖

Python/Typescript + OpenAI SDK

- Código inicial no chat
- Eu codifico, vocês sugerem ajustes
- 30 minutos colaborativos

Principais Aprendizados

- Contexto é rei
- Workflows primeiro, agents depois
- MCP é seu catálogo de ferramentas
- Cursor turbina seu código
- **Desafio:** o que você vai testar?
- Como eu posso ajudar a tirar todas as duvidas para começarmos a aplicar?

Suas Dúvidas 🧐👉

Pra Aprofundar

- Anthropic Workflows Cookbook
- OpenAI Agents SDK
- 12-Factor Agents by HumanLayer
- Documentação do Cursor
- Prompt Lib Business: <https://x.com/ridbay/status/1931405358519435766>
- ChatGPT Natural Prompt: <https://x.com/markgadala/status/1930637344790421785>